

Warehouse Location Scan — Build Specification

A new scan-to-locate app built on the same architecture as ZSOL_ISCAN (classic Gateway OData V2), so it works with the handheld scanner your dispatch team already uses. Scan a location barcode, then a pallet/carton barcode, and the system records which warehouse location that pallet/carton is currently in. Generated 26 Aug 2026.

What this does and doesn't do

Per your decisions: this is **location tracking only** — it records "this pallet/carton is now at Location X" and keeps a scan history for traceability. It does **not** post a stock transfer or change quantities/storage location in standard inventory — that stays a separate, larger decision if you want it later (it would mean mapping to a real goods movement, the way ZSOL_HU_ISCAN already does for HU transfers). Barcodes are plain generated codes (Code128) for you to print — not tied to a label-printer integration yet.

A tooling limit, up front

The ADT bridge I have access to can create and edit ABAP programs, classes, interfaces, and function groups — but not database tables, CDS views, or Gateway Service Builder (SEGW) projects. Those need SE11 and SEGW, which means a person with developer access has to run three short steps by hand. Everything below is written so that is a copy-paste job, not a design job — the field lists, the service wiring, and the actual class logic are all ready to use, following the exact pattern ZSOL_DISPATCH_ISCAN already uses successfully in your system.

1. Data model — create these 3 tables in SE11

Same package as the existing scan app: **ZSOL_ISCAN**. Same transport process you already use for that package.

ZSOL_LOC_MAST — Location Master

One row per defined warehouse location/bin. This is what the location barcode encodes.

Field	Type	Key?	Meaning
MANDT	MANDT	Key	Client
WERKS	WERKS_D	Key	Plant
LGORT	LGORT_D	Key	Storage location
LOC_CODE	CHAR(10)	Key	Location/bin code, e.g. PAST-A1
LOC_DESC	CHAR(40)		Description, e.g. "Pallet Store — Aisle 1"
ACTIVE	CHAR(1)		'X' = active, blank = retired
ERNAM	SYUNAME		Created by
ERDAT	ERDAT		Created on

ZSOL_BOX_LOC_CUR — Current location per box/pallet

One row per box/pallet (keyed on the same BOXNO type ZPP_PACK and ZSOL_HUDISPATCH already use). Every new scan overwrites this row — it always answers "where is this pallet right now".

Field	Type	Key?	Meaning
MANDT	MANDT	Key	Client
BOXNO	ZBOXNO	Key	Box/pallet/carton number
WERKS	WERKS_D		Plant of current location
LGORT	LGORT_D		Storage location of current location
LOC_CODE	CHAR(10)		Current location/bin code
SCAN_DATE	DATS		Date of this scan
SCAN_TIME	TIMS		Time of this scan
SCAN_USER	SYUNAME		Who scanned it

ZSOL_BOX_LOC_LOG — Scan history (append-only)

Every scan ever made, kept for traceability — "show me everywhere this pallet has been." Never updated, only appended to.

Field	Type	Key?	Meaning
MANDT	MANDT	Key	Client
BOXNO	ZBOXNO	Key	Box/pallet/carton number
SEQ_NO	NUMC(6)	Key	Sequence per box — increment per scan

Field	Type	Key?	Meaning
WERKS	WERKS_D		Plant scanned into
LGORT	LGORT_D		Storage location scanned into
LOC_CODE	CHAR(10)		Location/bin code scanned into
PREV_LOC_CODE	CHAR(10)		Location it moved from (blank on first scan)
SCAN_DATE	DATS		Date
SCAN_TIME	TIMS		Time
SCAN_USER	SYUNAME		Who scanned it

2. Gateway service — build in SEGW

New SEGW project **ZSOL_LOC_ISCAN**, same pattern as the existing ZSOL_DISPATCH_ISCAN project. Two entity sets:

Entity	Type	Purpose
LocationSet	Read-only, from ZSOL_LOC_MAST	Lets the scanner app validate a scanned location barcode exists and is active before accepting the second scan. Key: Werks, Lgort, LocCode.
BoxLocationScanSet	Deep-entity create, no backing table (processed in the DPC_EXT class)	The actual scan submission: LocCode + Werks + Lgort + Boxno. On create, it validates the location, reads the box's previous location (if any) from ZSOL_BOX_LOC_CUR, writes the new current location, and appends a row to ZSOL_BOX_LOC_LOG.

In the Service Builder: generate the runtime objects, then redefine **CREATE_DEEP_ENTITY** for **BoxLocationScanSet** in the generated **ZCL_ZSOL_LOC_ISCAN_DPC_EXT** class — this mirrors exactly how ZCL_ZSOL_DISPATCH_ISCA_DPC_EXT redefines the same method for its IscanSet entity. Register the service in /IWFND/MAINT_SERVICE and add its System Alias, same as the other ZSOL_ISCAN services.

3. DPC_EXT class — ready to paste

This is the actual scan logic. Once the SEGW project generates ZCL_ZSOL_LOC_ISCAN_DPC_EXT, replace the generated CREATE_DEEP_ENTITY stub with this:

```
METHOD /iwbep/if_mgw_appl_srv_runtime~create_deep_entity.
```

```
DATA: ls_scan      TYPE zcl_zsol_loc_iscan_mpc_ext=>ts_boxlocationscan,
      ls_loc       TYPE zsol_loc_mast,
      ls_cur       TYPE zsol_box_loc_cur,
      ls_log       TYPE zsol_box_loc_log,
      lv_prev_loc  TYPE char10,
      lv_next_seq  TYPE numc6,
      lv_max_seq   TYPE numc6.

CASE iv_entity_set_name.
  WHEN 'BoxLocationScanSet'.

    io_data_provider->read_entry_data(
      IMPORTING es_data = ls_scan ).

    " 1. validate the scanned location exists and is active
    SELECT SINGLE * FROM zsol_loc_mast
      INTO ls_loc
      WHERE werks = ls_scan-werks
        AND lgort = ls_scan-lgort
        AND loc_code = ls_scan-loccode
        AND active = 'X'.

    IF sy-subrc <> 0.
      RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
        EXPORTING
          textid = /iwbep/cx_mgw_busi_exception=>business_error
          message = |Location { ls_scan-loccode } not found or inactive for { ls_scan-werks }/{ ls_scan-lgort }|.
    ENDIF.

    " 2. read the box's previous location, if any
    SELECT SINGLE loc_code FROM zsol_box_loc_cur
      INTO lv_prev_loc
      WHERE boxno = ls_scan-boxno.

    " 3. upsert current location
    ls_cur-boxno = ls_scan-boxno.
    ls_cur-werks = ls_scan-werks.
    ls_cur-lgort = ls_scan-lgort.
    ls_cur-loc_code = ls_scan-loccode.
    ls_cur-scan_date = sy-datum.
    ls_cur-scan_time = sy-uzeit.
    ls_cur-scan_user = sy-uname.
    MODIFY zsol_box_loc_cur FROM ls_cur.

    " 4. append to history log
    SELECT MAX( seq_no ) FROM zsol_box_loc_log
      INTO lv_max_seq
      WHERE boxno = ls_scan-boxno.
    lv_next_seq = lv_max_seq + 1.

    ls_log-boxno = ls_scan-boxno.
    ls_log-seq_no = lv_next_seq.
    ls_log-werks = ls_scan-werks.
    ls_log-lgort = ls_scan-lgort.
    ls_log-loc_code = ls_scan-loccode.
    ls_log-prev_loc_code = lv_prev_loc.
    ls_log-scan_date = sy-datum.
    ls_log-scan_time = sy-uzeit.
    ls_log-scan_user = sy-uname.
    INSERT zsol_box_loc_log FROM ls_log.

    set_etag( iv_value = |Box { ls_scan-boxno } now at { ls_scan-loccode }| ).

    copy_data_to_ref(
```

```
EXPORTING is_data = ls_scan  
CHANGING cr_data = er_deep_entity ).
```

```
ENDCASE.
```

```
ENDMETHOD.
```

Same defensive style as the existing ZSOL_ISCAN classes: read, validate, write, confirm via the ETag message the scanner app displays. No BAPI or stock posting involved, matching the "tracking only" decision.

4. Scan flow on the handheld device

Step	Action	What happens
1	Scan the location barcode	App reads Werks/Lgort/LocCode from the barcode and calls LocationSet to confirm it exists and is active.
2	Scan the pallet/carton barcode	App reads Boxno.
3	App calls BoxLocationScanSet (create)	Server validates, updates ZSOL_BOX_LOC_CUR, appends to ZSOL_BOX_LOC_LOG.
4	App shows confirmation	"Box 9960000123 now at PAST-A1" or the business-exception message if the location was invalid/inactive.

This is the same two-scan pattern already used for dispatch box scanning (ZSOL_DISPATCH_ISCAN) — location first, then item — so it should feel identical to your team on the same device.

Next steps: (1) create the 3 tables in SE11 using the field lists above, in package ZSOL_ISCAN; (2) build the ZSOL_LOC_ISCAN SEGW project with the two entities described, generate runtime objects, register the service; (3) paste the class logic in section 3 into the generated DPC_EXT; (4) populate ZSOL_LOC_MAST with your real location codes — see the accompanying barcode PDF for a starter set drawn from your existing plant/storage-location combinations; (5) test on the handheld device the same way ZSOL_DISPATCH_ISCAN was tested.